Document number: P2076R0
Audience: SG21

Ville Voutilainen
2020-01-13

Previous disagreements on Contracts

Abstract

This paper attempts to recapture, remind, and to a limited extent, explain various earlier disagreements over what Contracts should or should not do and how. Here's a teaser trailer of those things:

- continuation
- build levels
- global toggles aka external remapping controls
- literal semantics aka in-source controls
- assumption

The limited-extent explanations are marked as "Rumination". They are brief overviews with the best intent to be just explanatory, and may be overly brief or not completely accurate, caveat emptor.

Continuation

We had a global continuation toggle in the C++2a WP, and more local ones (check_maybe_continue, check_never_continue) in various other proposals. There was also discussion about removing continuation in all of its forms.

Rumination

There was thus a two-part disagreement here, the parts being

1. whether a continuation facility should exist at all
2. whether it should be global or local (or both)

This disagreement certainly relates to the later-explained disagreement about global vs. local control. If the need for local control is deemed plausible, a global continuation flag doesn't match that need and the related use cases all that well. In contrast, if a preference for global toggles without local overrides is deemed plausible, a local control for continuation doesn't mesh with that all that well. Furthermore, if it's plausible that contracts should be 'hard truths', the ability to continue after a contract is not satisfied doesn't necessarily seem plausible.

(Yes, I know, that's a lot of ifs.) The other viewpoint is that it can arguably be very useful to write contract checks that do not abort or do not cause optimizations to be triggered.

Build levels

The C++2a WP contracts had global build levels that attempted to provide a linear scale. There was disagreement on whether a linear scale is feasible, and whether global build levels should be the only way to affect which contracts are checked.

Rumination

As an alternative to global build levels, we had proposals providing either additionally to or instead of a global build level setting, a perhaps more fine-grained or more semantic approach for it.

The debate is at least to some extent about simplicity versus applicability. In theory at least, some amounts of the debate are about timing, what goes first, and what the defaults are, but that's not all of it, as the simplicity argument might suggest that there shouldn't be multiple ways to control how contracts behave. The counterargument is that local or semantic controls can do things that build levels can't.

Global toggles aka external remapping controls

The C++2a WP contracts had all the contract semantics setting and enabling/disabling as global controls. Alternative proposals allowed for local control.

Rumination

There was disagreement on whether a global contract enable/disable should affect all contracts, or whether some contracts could override or be unaffected by such global settings.

This is in a sense repeating the simplicity versus applicability theme, but there are other twists in it; global vs. local remapping, or even the ability to decide what can or cannot be remapped on any which level, can potentially have a big impact on how code using contracts is written, deployed, integrated to other code, and debugged.

Literal semantics aka in-source controls

Various proposals allowed setting per-contract contract semantics in source code, and especially so that those semantics would not be remappable or otherwise affected by any global or external controls or toggles.

Rumination

There was certainly disagreement on whether such ideas were mature enough to be incorporated into C++20. There's also been disagreement on how fine-grained the contract semantics control should be, and we have had many discussions about various aspects of it, such as simplicity and applicability.

The other rumination bits for build levels and global controls apply here as well.

Assumption

The C++2a WP contracts ended up with unchecked contracts being assumed. It was heavily debated whether this did or did not match the original design intent.

Rumination

This has been one of the biggest disagremeents related to contracts, although not necessarily bigger than global vs. local control, overall. But it's fair to note that the local control proposals sought to avoid the problems with undesired assumptions as well.

There was disagreement on whether assuming unchecked contracts is reasonable; arguments have been provided both ways. Some part of the disagreement is also about timing and defaults.

A desperate attempt at a conclusion, and a homework suggestion

When we discuss what Contracts in C++2b should look like and what they should do and be able to do, we should pay heed to these earlier disagreements. We should also keep in mind that timing and schedule are now different, at least somewhat, if not completely so. At any rate, we should be aware of these disagreements and try to address them early, or at least try and avoid ending up with very-late impasses like we did with C++2a Contracts.

The actual homework suggestion for SG21 members is thus:

1. Look at the contracts requirements and their priorities with these disagreements in mind.
2. When designing C++2b Contracts and writing proposals for them, consider these disagreements and figure out ways to avoid dead-end disagreements.
3. Look at the arguments of both sides of these disagreements with a critical eye and a cool head.
4. Perhaps most importantly, be very careful about arguments like "the facility can NEVER support $foo" and "the facility MUST have $bar in its first incarnation". Both of those arguments lead to an end result where the first argument's wish is granted, but nothing else is provided either, and the second argument's wish isn't granted, and overall, nobody gains anything and we are unequally unhappy, and have failed to serve all users, except users who want the language to stop evolving.